

Day 2 - Python Functions, Lists, Comprehensions & Dictionaries

Date: 2026-09-14

Purpose: Detailed revision archive of the concepts, actual code, outputs, mistakes, corrections, and engineering connections covered today. This includes the code written in the Daily_Code_Book and the reference snippets used during teaching.

Day 2 result: Strong. The learner repeatedly predicted behavior correctly and independently produced working engineering-oriented programs.

Next day starts with: Recall test before new material.

1. Day 2 Recall Gate

The opening recall covered variables, = vs ==, input() returning str, range(), break vs continue, and if/elif/else. All six answered questions were correct. Day 1 homework, Engineering Current Monitor, was also executed correctly.

2. Function Basics

Function: a named, reusable block of instructions. Define once; call many times. A function is analogous to a reusable engineering calculation/subcircuit.

```
def greet():  
    print("Hello")
```

```
greet()  
greet()  
greet()
```

Defining a function does not execute it. The calls execute it.

3. Parameters, Arguments, and return

```
def calculate_current(voltage, resistance):  
    return voltage / resistance
```

```
current1 = calculate_current(12, 1000)  
current2 = calculate_current(24, 2000)  
current3 = calculate_current(5, 500)
```

Parameter: placeholder in the definition. **Argument:** actual value supplied at the call. **return:** sends a result back to the caller and exits the function.

Calculated: current1 = 0.012 A, current2 = 0.012 A, current3 = 0.01 A.

4. User's Interactive Resistance Calculator

The learner independently improved the fixed-value exercise by accepting arbitrary measured inputs.

```
def calculate_resistance(voltage, current):  
    return voltage / current
```

```
voltage_input = float(input("Please enter your measured voltage: "))  
current_input = float(input("Please enter your measured current: "))  
calculated_res = calculate_resistance(voltage_input, current_input)  
print(calculated_res)
```

Observed tests: 12/2 -> 6.0, 24/3 -> 8.0, 5/0.5 -> 10.0.

Feedback: Good engineering design instinct. Future robustness issue identified: current = 0 would cause division by zero.

5. print vs return

```
def calculate_power(voltage, current):  
    return voltage * current
```

```
power = calculate_power(12, 2)
print(power)
```

print() displays a value. **return** hands a value back to the calling code, where it can be stored, compared, or reused.

6. Local Scope

```
def calculate_power(voltage, current):
    power = voltage * current
    return power

result = calculate_power(10, 2)
print(result)
```

power is local to the function. result is outside the function and receives the returned value.

7. Default Parameters

```
def calculate_power(voltage, current, efficiency=1.0):
    return voltage * current * efficiency

power1 = calculate_power(10, 2)
power2 = calculate_power(10, 2, 0.8)

print(power1)
print(power2)
```

Output: 20 and 16. efficiency=1.0 is the default; a supplied 0.8 overrides it.

8. Positional and Keyword Arguments

```
def calculate_power(voltage, current, efficiency=1.0):
    return voltage * current * efficiency

a = calculate_power(12, 2)
b = calculate_power(current=2, voltage=12)
c = calculate_power(2, 12)
```

a = 24, b = 24, c = 24. b uses keyword arguments, so order is irrelevant. c happens to equal 24 because multiplication is commutative.

```
def calculate_current(voltage, resistance):
    return voltage / resistance

calculate_current(12, 2000)
calculate_current(2000, 12)
```

For division, swapping positional arguments changes the result drastically; parameter order matters.

9. Multiple Return Values and Unpacking

```
def analyze_measurement(voltage, current):
    power = voltage * current
    resistance = voltage / current
    return power, resistance

power, resistance = analyze_measurement(12, 2)
print(power)
print(resistance)
```

Output: 24 and 6.0. Returning multiple comma-separated values and assigning them to multiple variables is commonly called tuple unpacking.

10. Boolean-returning Function

```
def is_safe_voltage(voltage):
    return 10 <= voltage <= 20

voltage = 15

if is_safe_voltage(voltage):
    print("NORMAL")
else:
```

```
print("UNSAFE")
```

A function can answer a yes/no engineering question with True or False, leaving the caller to decide what action to take.

11. Current Classification Function - User's Code

```
def classify_current(current):
    if current < 1:
        return "LOW"
    elif 1 <= current <= 3:
        return "NORMAL"
    elif 3 < current < 4:
        return "HIGH"
    else:
        return "FAULT"

current_data = [0.5, 2.0, 3.5, 4.2]
for current in current_data:
    print(classify_current(current))
```

Output: LOW, NORMAL, HIGH, FAULT. Feedback: correct. Good reuse of one function over many values.

12. Lists as Function Inputs

```
def find_max_voltage(voltages):
    max_voltage = voltages[0]

    for voltage in voltages:
        if voltage > max_voltage:
            max_voltage = voltage

    return max_voltage

voltages = [12, 18, 15, 21, 17]
print(find_max_voltage(voltages))
```

Important correction from the first attempt: the running maximum had been initialized inside the loop, so it was reset every iteration. Moving initialization before the loop lets state persist between iterations. The corrected output is 21.

13. len(), range(), Indexing, and In-place List Modification

```
def voltage_add(voltages):
    for i in range(len(voltages)):
        voltages[i] = voltages[i] + 2

voltages = [10, 12, 14, 16]
voltage_add(voltages)
print(voltages)
```

Output: [12, 14, 16, 18]. len() gives the count of elements. range(len(...)) produces valid indexes. voltages[i] accesses an element by index. Assignment to voltages[i] changes that element.

14. Mutable vs Immutable Objects

```
def change_voltage(voltages):
    voltages[0] = 100

voltages = [10, 20, 30]
change_voltage(voltages)
print(voltages)
```

Output: [100, 20, 30]. A list is mutable, so the function can modify the existing list object.

```
def change_voltage(voltage):
    voltage = 100

voltage = 10
change_voltage(voltage)
print(voltage)
```

Output: 10. An integer is immutable; the local assignment does not modify the outside variable.

```
def change_name(name):
    name = "Texas Instruments"
```

```
name = "Harsh"
change_name(name)
print(name)
```

Output: Harsh. Strings are immutable.

15. Gain Processing - User's Code

```
def apply_gain(measurements, gain):
    for i in range(len(measurements)):
        measurements[i] = measurements[i] * gain

measurements = [1.0, 2.0, 3.0, 4.0]
gain = 2.5
apply_gain(measurements, gain)
print(measurements)
```

Output: [2.5, 5.0, 7.5, 10.0]. This modifies the original list in place.

16. Creating a New List and Preserving Raw Data

```
def apply_offset(measurements, offset):
    processed = []
    for measurement in measurements:
        processed.append(measurement + offset)
    return processed

measurements = [10.0, 12.5, 15.0, 17.5]
offset = 0.5
processed_measurements = apply_offset(measurements, offset)

print(measurements)
print(processed_measurements)
```

The original list stays unchanged. This is often preferable in engineering data processing because raw measurements should be preserved.

17. List Comprehensions

```
processed = [measurement + offset for measurement in measurements]
```

General pattern: `new_list = [expression for item in old_list]`.

```
currents = [0.5, 1.2, 2.5, 3.1, 4.0]
safe_currents = [current for current in currents if current <= 3]
```

Filtering pattern: `[expression for item in collection if condition]`. Result: [0.5, 1.2, 2.5].

18. User's Function + List-Comprehension Code

```
def apply_offset(measurements, offset):
    processed = [measurement + offset for measurement in measurements]
    return processed

measurements = [10.0, 12.5, 15.0, 17.5]
offset = 0.5
processed_measurements = apply_offset(measurements, offset)
print(measurements)
print(processed_measurements)
```

Output: [10.0, 12.5, 15.0, 17.5] and [10.5, 13.0, 15.5, 18.0]. Feedback: the learner independently used list-comprehension syntax; this was a positive sign of understanding.

19. User's Conditional List-Comprehension Function

```
def current_3(currents):
    processed = [current for current in currents if current <= 3]
    return processed

currents = [0.5, 1.2, 2.5, 3.1, 4.0]
processed = current_3(currents)
print(currents)
print(processed)
```

Output: original currents unchanged; processed = [0.5, 1.2, 2.5]. Feedback: function works; a clearer name such as filter_safe_currents would be easier for a future teammate to understand.

20. Dictionaries and Key-Value Pairs

```
device = {
    "part_number": "LMG2107",
    "max_voltage": 100,
    "max_current": 4
}

print(device["part_number"])
print(device["max_voltage"])
```

A dictionary stores key-value pairs. Example: key = "max_current", value = 4. Square brackets retrieve a value by key.

21. Nested Dictionaries

```
device = {
    "part_number": "LMG2107",
    "electrical": {
        "max_voltage": 100,
        "max_current": 4
    }
}

print(device["electrical"]["max_voltage"])
print(device["electrical"]["max_current"])
```

The first lookup gets the nested electrical dictionary. The second lookup gets the specific specification within it.

22. Dictionary Iteration with .items()

```
specs = {
    "max_voltage": 100,
    "max_current": 4,
    "max_temperature": 150
}

for name, value in specs.items():
    print(name, value)
```

items() supplies key-value pairs; each pair is unpacked into name and value. Output lines correspond to each specification.

23. Voltage Validation Using Structured Device Data

```
def check_voltage(device, measured_voltage):
    if measured_voltage <= device["electrical"]["max_voltage"]:
        return "SAFE"
    else:
        return "OVER VOLTAGE"

result = check_voltage(device, 80)
print(result)
```

This demonstrates data -> reusable function -> engineering decision. The same structure is useful for device-specification validation.

24. Final Day 2 Exercise - User's Device Checker

```
def check_device(device, measurements):
    if device["max_voltage"] > measurements["voltage"]:
        print("VOLTAGE_OK")
    elif device["max_voltage"] < measurements["voltage"]:
        print("VOLTAGE_FAULT")
    if device["max_current"] > measurements["current"]:
        print("CURRENT_OK")
    else:
        print("CURRENT_FAULT")

device = {
    "part_number": "LMG2107",
    "max_voltage": 100,
```

```
        "max_current": 4
    }
    measurements = {
        "voltage": 85,
        "current": 3.2
    }

    check_device(device, measurements)
```

Observed output: VOLTAGE_OK and CURRENT_OK.

Feedback: The tested case is correct. One boundary bug remains: at exactly 100 V, neither voltage branch runs. A robust version should include the equality case in the safe branch and use an else for the fault case.

25. Engineering Patterns Learned

- Reusable calculation: inputs -> function -> returned result.
- Reusable validation: structured data -> function -> engineering decision.
- Stateful loop algorithm: initialize -> inspect -> update -> return.
- Raw vs processed data: preserve raw measurement lists when transformations should be non-destructive.
- Structured specifications: dictionaries and nested dictionaries map naturally to device/datasheet fields.
- The concepts learned today will later support waveform processing, SPICE/PSpice automation, datasheet extraction, structured LLM outputs, and AI/EDA tooling.

26. Day 2 Mastery / Weaknesses

- Strong: function mechanics, return values, loops, list processing, dictionary access, and independent coding.
- Strong: reasoning about code before execution.
- Needs reinforcement: scope and mutable/immutable behavior.
- Needs reinforcement: edge-case validation, especially equality boundaries and division by zero.
- Needs improvement: naming clarity (for example, max_voltage instead of present_int; descriptive function names instead of current_3).

27. Resume From Here

Next session: begin with a recall test. Then continue Python foundations toward robust input/error handling and stronger software-engineering habits.

Code archive: Daily_Code_Book/Day_2_functions.py

Records to push: progress.md, study_diary/Day_02_Python_Functions_Diary.md, study_diary/Day_02_Python_Functions_and_Data_Revision_Book.pdf